Praktikum JavaScript & TypeScript

Modul 3 - Array dan Method Array

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, peserta mampu:

- Menyimpan dan membaca data berbentuk array of object
- Menjelaskan cara kerja callback: siapa yang memanggilnya dan apa isi parameternya
- Memilih method yang tepat (`forEach`, `map`, `filter`, `find`, `some`, `every`, `includes`, `reduce`, `join`, `slice`, `sort`) dari pertanyaan yang ingin dijawab
- Merangkai beberapa method menjadi satu alur (*chaining*)
- Menggabungkan dua array lewat id, seperti JOIN di database
- Membaca pesan error yang paling sering muncul saat memakai method array

2. Dasar Teori

2.1. Array of Object

Data nyata jarang berupa angka saja. Satu pasien punya nama, umur, dan poli. Data seperti ini disimpan dalam *object*. Banyak object disimpan dalam satu array.

Jika kamu terbiasa dengan database, bayangkan seperti ini:

- Array adalah **tabel**.
- Setiap object adalah satu **baris**.
- Setiap properti (`nama`, `umur`, `poli`) adalah satu **kolom**.

```
const pasien = [
  { nama: "Budi", umur: 40, poli: "Umum" },   // index 0
  { nama: "Sari", umur: 7, poli: "Anak" },    // index 1
  { nama: "Joko", umur: 65, poli: "Umum" },   // index 2
];
console.log(pasien.length);    // 3      (banyak baris)
console.log(pasien[1]);        // { nama: "Sari", umur: 7, poli: "Anak" }
console.log(pasien[1].nama);   // "Sari" (baris index 1, kolom nama)
```

Listing 1: array of object

Baca `pasien[1].nama` dari kiri ke kanan: ambil array `pasien`, ambil elemen index 1, lalu ambil properti `nama` dari elemen itu.

> ⚠ **Perhatian**
>
> Properti `nama` dimiliki setiap **elemen**, bukan array-nya. `pasien.nama` menghasilkan `undefined`, karena array tidak punya kolom bernama `nama`. Ini kesalahan yang paling sering terjadi di modul ini. Sebelum menulis `x.nama`, tanyakan dulu: `x` ini satu object, atau seluruh array?

Object dibahas lebih dalam di Modul 4. Di modul ini cukup tahu cara membuat object dan membaca propertinya.

2.2. Callback: Isi Perulangan yang Kamu Titipkan

Semua method di modul ini adalah **perulangan yang sudah jadi**. Kamu tidak menulis `for` sendiri. Kamu hanya menulis **isi perulangannya** sebagai fungsi, lalu menitipkannya ke method. Fungsi titipan ini disebut *callback*.

Dua kode berikut melakukan hal yang sama:

```
// Cara Modul 2: kamu menulis perulangannya sendiri
for (const p of pasien) {
  console.log(p.nama);
}

// Cara Modul 3: forEach yang mengulang, kamu hanya memberi isinya
pasien.forEach((p) => {
  console.log(p.nama);
});
```

Listing 2: for...of dan forEach

Hal penting tentang callback:

1. **Method yang memanggil callback**, bukan kamu. Callback dipanggil sekali untuk setiap elemen, dari index 0 sampai terakhir.
2. **Method yang mengisi parameternya.** Parameter pertama selalu berisi elemen yang sedang diproses, sama seperti `p` pada `for (const p of pasien)`. Parameter kedua (opsional) berisi index-nya.
3. **Nama parameter bebas.** `(p) => ...`, `(pasien) => ...`, dan `(x) => ...` sama saja. Pilih nama yang menjelaskan isinya. Satu pasien cukup `p`, satu item keranjang cukup `item`.
4. **Nilai yang di-`return` callback dipakai oleh method.** Artinya beda-beda per method. `map` menyimpannya, `filter` memakainya sebagai keputusan lolos atau tidak, `reduce` membawanya ke putaran berikutnya.

> **⚠ Perhatian**
>
> Ingat Modul 2: arrow function dengan kurung kurawal `{ }` wajib memakai `return`. `(p) => p.nama` mengembalikan nama. `(p) => { p.nama }` mengembalikan `undefined`.

2.3. Memilih Method dari Pertanyaannya

Cara tercepat memilih method: tulis dulu **pertanyaan** yang ingin dijawab, lalu cocokkan dengan tabel berikut. Kolom SQL hanya pembanding.

Method	Pertanyaan yang dijawab	Hasil	Mirip SQL
`forEach`	Lakukan sesuatu untuk setiap elemen	`undefined`	-
`map`	Ubah setiap elemen menjadi apa?	array baru, panjang **sama**	`SELECT kolom`
`filter`	Elemen mana saja yang memenuhi syarat?	array baru, bisa lebih pendek	`WHERE`
`find`	Elemen **pertama** mana yang memenuhi syarat?	**satu** elemen atau `undefined`	`WHERE ... LIMIT 1`
`findIndex`	Di index berapa elemen itu?	number, atau `-1`	-
`some`	**Adakah** elemen yang memenuhi syarat?	satu boolean	`EXISTS`
`every`	Apakah **semua** elemen memenuhi syarat?	satu boolean	-
`includes`	Apakah nilai ini ada di array?	satu boolean	`IN (...)`
`reduce`	Gabungkan semua elemen menjadi satu nilai	satu nilai	`SUM`, `COUNT`

`join`	Gabungkan semua elemen menjadi satu string	string	-
`slice`	Ambil sebagian elemen	array baru	`LIMIT`
`sort`	Urutkan elemen	array **yang sama**	`ORDER BY`

Tidak ada method untuk `JOIN` dan `GROUP BY`. Keduanya disusun dari method di atas, dibahas di bagian "Menggabungkan Dua Array lewat id" dan "Mengelompokkan Data".

Semua method di tabel ini **tidak mengubah array asli**, kecuali `sort`. `sort` dibahas di bagian tersendiri.

Perhatikan kolom Hasil. Kolom itu menentukan apa yang boleh ditulis sesudahnya. Hasil berupa array bisa dilanjutkan dengan method array lain. Hasil berupa satu object, boolean, number, atau string tidak bisa.

2.3.1. forEach: lakukan aksi untuk setiap elemen

Untuk apa: menjalankan aksi, misalnya mencetak atau mengirim data, untuk setiap elemen. `forEach` tidak menghasilkan data baru.

Cara kerja: panggil callback untuk elemen index 0, 1, 2, dan seterusnya. Nilai `return` callback dibuang.

```
pasien.forEach((p, i) => {
  console.log(`${i + 1}. ${p.nama}`);
});
// 1. Budi
// 2. Sari
// 3. Joko

const hasil = pasien.forEach((p) => p.nama);
console.log(hasil);   // undefined, karena forEach tidak mengembalikan apa-apa
```

Listing 3: forEach

Kesalahan umum: menyimpan hasil `forEach` ke variabel. Hasilnya selalu `undefined`. Jika butuh data baru, pakai `map` atau `filter`.

2.3.2. map: ubah setiap elemen

Untuk apa: membuat array baru yang isinya hasil olahan setiap elemen. Contohnya mengambil satu kolom, menghitung nilai baru, atau menyusun teks.

Cara kerja: untuk setiap elemen, panggil callback, lalu simpan nilai `return`-nya ke array baru. Panjang array baru **selalu sama** dengan array asal. Tiga pasien menghasilkan tiga elemen, apa pun isi callback-nya.

Versi `for` dari `map` membantu melihat apa yang terjadi di dalamnya:

```
const namaSaja = pasien.map((p) => p.nama);
// ["Budi", "Sari", "Joko"]

// Yang dilakukan map di dalamnya kira-kira seperti ini:
const namaSaja2 = [];
for (const p of pasien) {
  namaSaja2.push(p.nama);   // nilai return callback disimpan
}
```

Listing 4: map dan versi for-nya

Jalannya `pasien.map((p) => p.nama)` per putaran:

Putaran	Isi `p`	Callback mengembalikan	Array baru
1	`{ nama: "Budi", umur: 40, ... }`	`"Budi"`	`["Budi"]`
2	`{ nama: "Sari", umur: 7, ... }`	`"Sari"`	`["Budi", "Sari"]`

3	`{ nama: "Joko", umur: 65, ... }`	`"Joko"`	`["Budi", "Sari", "Joko"]`

Callback boleh berisi beberapa baris, asalkan diakhiri `return`. Untuk menampilkan beberapa nilai sekaligus, gabungkan dengan template literal menjadi satu string:

```
const tagihan = [
  { nama: "Budi", hari: 3, tarif: 250000 },
  { nama: "Sari", hari: 1, tarif: 400000 },
];
const rincian = tagihan.map((t) => {
  const total = t.hari * t.tarif;              // hitung dulu
  return `${t.nama} ${t.hari} hari = ${total}`; // satu string, banyak nilai
});
// ["Budi 3 hari = 750000", "Sari 1 hari = 400000"]
```

Listing 5: map dengan callback beberapa baris

Kesalahan umum:

- Memakai `map` untuk menyaring. `pasien.map((p) => p.umur > 18)` menghasilkan `[true, false, true]`, bukan daftar pasien dewasa. `map` tidak pernah membuang elemen. Untuk menyaring, pakai `filter`.
- Lupa `return` di callback berkurung kurawal. Hasilnya `[undefined, undefined, undefined]`.

2.3.3. filter: saring elemen

Untuk apa: mengambil elemen yang memenuhi syarat dan membuang sisanya.

Cara kerja: untuk setiap elemen, panggil callback. Callback harus mengembalikan `true` atau `false`. Jika `true`, elemen **aslinya** dimasukkan ke array baru. Jika `false`, elemen dilewati. Callback `filter` adalah kondisi `WHERE`.

```
const pasienUmum = pasien.filter((p) => p.poli === "Umum");
// [{ nama: "Budi", ... }, { nama: "Joko", ... }]

// Versi for:
const pasienUmum2 = [];
for (const p of pasien) {
  if (p.poli === "Umum") pasienUmum2.push(p);   // yang disimpan elemennya, bukan true
}

const kosong = pasien.filter((p) => p.umur > 100);
// [] (array kosong, bukan null dan bukan error)
```

Listing 6: filter dan versi for-nya

Hasil `filter` tetap array object yang utuh. Jika hanya butuh namanya, lanjutkan dengan `map` (lihat bagian Chaining).

Kesalahan umum: mengira hasil `filter` adalah satu object. `pasien.filter((p) => p.poli === "Anak")` menghasilkan array berisi satu object, bukan object-nya. Jika memang hanya butuh satu, pakai `find`.

2.3.4. find dan findIndex: cari satu elemen

Untuk apa: mencari **satu** elemen, biasanya berdasarkan id atau nilai yang unik.

Cara kerja: panggil callback mulai dari index 0. Begitu callback mengembalikan `true`, `find` **berhenti** dan mengembalikan elemen itu. Jika sampai akhir tidak ada yang cocok, hasilnya `undefined`. `findIndex` sama, tetapi mengembalikan index-nya, atau `-1` jika tidak ketemu.

```
const anak = pasien.find((p) => p.poli === "Anak");
console.log(anak);        // { nama: "Sari", umur: 7, poli: "Anak" }  <- object, bukan array
console.log(anak.nama);   // "Sari"

// Versi for:
function cariPoli(poli) {
  for (const p of pasien) {
    if (p.poli === poli) return p;   // berhenti di yang pertama cocok
  }
  return undefined;
}

pasien.findIndex((p) => p.poli === "Anak");   // 1

const gigi = pasien.find((p) => p.poli === "Gigi");
console.log(gigi);        // undefined
console.log(gigi.nama);   // TypeError: Cannot read properties of undefined (reading 'nama')
```

Listing 7: find dan versi for-nya

Kesalahan umum:

- Memanggil method array pada hasil `find`, misalnya `pasien.find(...).map(...)`. Hasil `find` adalah satu object, bukan array, jadi tidak punya `map`. Ambil propertinya langsung: `pasien.find(...).nama`.
- Tidak mengecek `undefined`. Jika data yang dicari mungkin tidak ada, cek dulu hasilnya sebelum membaca properti.

2.3.5. some, every, includes: pertanyaan ya atau tidak

Untuk apa: menjawab pertanyaan ya atau tidak tentang array. Ketiganya menghasilkan **satu** boolean, bukan array boolean.

Method	Menghasilkan `true` jika	Berhenti lebih awal saat
`some(cb)`	minimal satu callback bernilai `true`	menemukan `true` pertama
`every(cb)`	semua callback bernilai `true`	menemukan `false` pertama
`includes(nilai)`	ada elemen yang `===` nilai itu	menemukan elemen yang sama

```
pasien.some((p) => p.umur >= 60);    // true  (Joko 65)
pasien.every((p) => p.umur >= 18);   // false (Sari 7)
pasien.map((p) => p.umur >= 18);     // [true, false, true]  <- array, bukan jawaban ya/tidak

const poliBuka = ["Umum", "Anak", "Gigi"];
poliBuka.includes("Anak");           // true
poliBuka.includes("Mata");           // false
```

Listing 8: some, every, includes

`includes` menerima **nilai**, bukan callback. Karena itu `includes` cocok untuk array berisi nilai sederhana seperti string atau number. Untuk array object, pakai `some` dengan syarat.

⚠ Perhatian

Array JavaScript **tidak** punya method `contains()`. Memanggilnya menghasilkan `TypeError: poliBuka.contains is not a function`. Nama `contains` berasal dari Java dan C#. Di JavaScript, `contains()` hanya ada di DOM, misalnya `element.classList.contains("aktif")`. Untuk array dan string, pakai `includes()`.

⚠ Perhatian

`includes` membandingkan dengan `===`. Untuk object, `===` membandingkan **alamat** object, bukan isinya. Jadi `pasien.includes({ nama: "Budi", umur: 40, poli: "Umum" })` menghasilkan `false`, karena object baru punya alamat berbeda. Untuk mencari object berdasarkan isinya, pakai `some` atau `find`.

2.3.6. reduce: gabungkan menjadi satu nilai

Untuk apa: mengubah seluruh array menjadi **satu** nilai. Contohnya total, jumlah, nilai terbesar, atau hitungan.

Cara kerja: `reduce(callback, nilaiAwal)` membawa satu variabel bernama *akumulator* dari putaran ke putaran. Callback menerima dua parameter: akumulator saat ini dan elemen saat ini. Nilai `return` callback menjadi akumulator untuk putaran berikutnya. Setelah elemen terakhir, akumulator itulah hasilnya.

Versi `for`-nya adalah pola "variabel di luar, diperbarui di dalam" dari Modul 2:

```
const biaya = [150000, 75000, 300000];
const total = biaya.reduce((acc, b) => acc + b, 0);   // 525000

// Versi for:
let acc = 0;                  // nilaiAwal
for (const b of biaya) {
  acc = acc + b;              // return callback menjadi acc baru
}
```

Listing 9: reduce dan versi for-nya

Jalannya `biaya.reduce((acc, b) => acc + b, 0)` per putaran:

Putaran	`acc` masuk	`b`	Callback mengembalikan
1	`0` (nilai awal)	`150000`	`150000`
2	`150000`	`75000`	`225000`
3	`225000`	`300000`	`525000` (hasil akhir)

Untuk array object, ambil propertinya di dalam callback. Beri nama parameter sesuai isinya: `acc` untuk total sementara, `p` untuk satu pasien.

```
const totalUmur = pasien.reduce((acc, p) => acc + p.umur, 0);   // 40 + 7 + 65 = 112
```

Listing 10: reduce pada array object

> **⚠ Perhatian**
>
> Selalu tulis nilai awal (argumen kedua). Tanpa nilai awal, `reduce` memakai elemen pertama sebagai akumulator dan mulai dari elemen kedua. Untuk array angka ini kebetulan benar. Untuk array object, akumulator pertama adalah **object**, sehingga `acc + p.umur` menyambung string `"[object Object]..."`. Untuk array kosong tanpa nilai awal, `reduce` melempar `TypeError: Reduce of empty array with no initial value`.

2.3.7. join dan slice

`join(pemisah)` menggabungkan semua elemen array menjadi **satu string**. Argumennya adalah teks pemisah, bukan data. `join` adalah method array, jadi dipanggil pada array, biasanya di ujung chaining.

`slice(awal, akhir)` mengambil sebagian elemen, dari index `awal` sampai **sebelum** index `akhir`. Hasilnya array baru, dan array asli tidak berubah. `slice(0, 3)` sama dengan `LIMIT 3`.

```
const nama = ["Budi", "Sari", "Joko"];
nama.join(", ");       // "Budi, Sari, Joko"
nama.join(" | ");      // "Budi | Sari | Joko"
nama.slice(0, 2);      // ["Budi", "Sari"]

"Budi".join(", ");     // TypeError: "Budi".join is not a function  <- string, bukan array
```

Listing 11: join dan slice

2.3.8. sort: urutkan elemen

Untuk apa: mengurutkan elemen, misalnya dari harga termurah.

Cara kerja: `sort` membandingkan dua elemen sekaligus, `a` dan `b`, lewat callback. Callback mengembalikan:

- angka **negatif** jika `a` harus di depan `b`,
- angka **positif** jika `b` harus di depan `a`,
- `0` jika urutannya sama saja.

Untuk angka, `a - b` otomatis memberi tanda yang tepat untuk urutan naik. `b - a` memberi urutan turun. Tanpa callback, `sort` mengubah elemen menjadi **string** dulu, lalu mengurutkannya seperti kamus.

```
[10, 9, 1].sort();                 // [1, 10, 9]  <- urut sebagai teks: "1" < "10" < "9"
[10, 9, 1].sort((a, b) => a - b);  // [1, 9, 10]  naik
[10, 9, 1].sort((a, b) => b - a);  // [10, 9, 1]  turun

const urutUmur = [...pasien].sort((a, b) => a.umur - b.umur);
// pasien asli tidak berubah, karena yang diurutkan salinannya
```

Listing 12: sort

Berbeda dari method lain di modul ini, `sort` mengurutkan array **asli** dan mengembalikan array yang sama. `[...pasien]` membuat salinan array (*spread*, dibahas di Modul 4). Node 20 ke atas juga punya `toSorted()` yang langsung mengembalikan array baru.

> **⚠ Perhatian**
>
> Mengubah array asli bisa merusak bagian program lain yang memakai array yang sama. Bug seperti ini sulit dilacak karena error tidak muncul di tempat perubahan terjadi. Biasakan menyalin array sebelum `sort`.

2.4. Chaining

Method yang menghasilkan array bisa langsung disambung dengan method berikutnya. Setiap method bekerja pada **hasil** method sebelumnya, bukan pada data asli.

```
const kunjungan = [
  { nama: "Budi", poli: "Umum", biaya: 150000 },
  { nama: "Sari", poli: "Anak", biaya: 200000 },
  { nama: "Joko", poli: "Umum", biaya: 90000 },
];
const totalUmum = kunjungan
  .filter((k) => k.poli === "Umum")       // WHERE poli = 'Umum'
  .map((k) => k.biaya)                    // SELECT biaya
  .reduce((acc, b) => acc + b, 0);        // SUM(biaya)
// 240000
```

Listing 13: chaining: total biaya pasien umum

Bentuk data berubah di setiap langkah. Selalu tahu bentuknya sebelum menulis langkah berikutnya:

Setelah	Bentuk data
`kunjungan`	array 3 object
`.filter(...)`	array 2 object (Budi, Joko), propertinya masih lengkap
`.map((k) => k.biaya)`	array 2 number: `[150000, 90000]`
`.reduce(...)`	satu number: `240000`. Rantai berakhir di sini karena number bukan array

Tiga aturan chaining:

1. **Urutan penting.** Di SQL, urutan eksekusi klausa sudah tetap. Di JS, urutan tulisan adalah urutan eksekusi. Jika `map((k) => k.biaya)` ditulis sebelum `filter`, properti `poli` sudah hilang dan `filter` tidak bisa menyaring.
2. **Saring dulu, baru ubah.** Biasanya `filter` di depan, `map` sesudahnya, dan `reduce` atau `join` di ujung.

3. **Cek setiap langkah.** Jika hasil akhir salah, potong rantainya dan `console.log` hasil setiap langkah.

2.5. Menggabungkan Dua Array lewat id

Sering ada dua array yang saling merujuk lewat id, seperti foreign key di database. Contohnya daftar dokter dan daftar kunjungan. Setiap kunjungan hanya menyimpan `kodeDokter`, sedangkan nama dokter ada di array `dokter`.

```
const dokter = [
  { kode: "D1", nama: "dr. Rina", poli: "Umum" },
  { kode: "D2", nama: "dr. Andi", poli: "Anak" },
];
const kunjungan = [
  { pasien: "Budi", kodeDokter: "D1" },
  { pasien: "Sari", kodeDokter: "D2" },
  { pasien: "Joko", kodeDokter: "D1" },
];
```

Listing 14: dua array yang saling merujuk

Di SQL cukup menulis `JOIN dokter ON dokter.kode = kunjungan.kodeDokter`. Database yang mengulang setiap baris kunjungan dan mencari pasangannya. Di JS, kedua langkah itu kamu tulis sendiri:

1. **Ulangi setiap kunjungan** dengan `map`, karena hasilnya satu baris per kunjungan.
2. **Di dalam callback** `map`, cari **satu** dokter yang `kode`-nya sama dengan `kodeDokter` kunjungan ini, memakai `find`.

```
const daftar = kunjungan.map((k) => {
  const d = dokter.find((dok) => dok.kode === k.kodeDokter);
  return `${k.pasien} diperiksa ${d.nama}`;
});
// ["Budi diperiksa dr. Rina", "Sari diperiksa dr. Andi", "Joko diperiksa dr. Rina"]
```

Listing 15: menggabungkan kunjungan dengan dokter

Perhatikan dua variabel di dalamnya:

- `k` adalah **satu kunjungan** yang sedang diproses `map`. Inilah “baris saat ini” yang di SQL ditulis `kunjungan.kodeDokter`. Di JS, `kunjungan.kodeDokter` bernilai `undefined`, karena `kunjungan` adalah seluruh array.
- `dok` adalah **satu dokter** yang sedang diperiksa `find`. Callback `find` dijalankan untuk setiap dokter sampai ketemu. Hasil `find`, yaitu dokter yang cocok, disimpan di `d`. Nama `dok` dan `k` sengaja dibuat berbeda supaya tidak tertukar.

Versi `for`-nya adalah dua perulangan bersarang, sama seperti cara kerja JOIN paling sederhana:

```
const daftar2 = [];
for (const k of kunjungan) {          // perulangan luar: setiap kunjungan
  let d;
  for (const calon of dokter) {       // perulangan dalam: cari dokternya (find)
    if (calon.kode === k.kodeDokter) {
      d = calon;
      break;
    }
  }
  daftar2.push(`${k.pasien} diperiksa ${d.nama}`);
}
```

Listing 16: versi for dari penggabungan

> **⚠ Perhatian**
>
> Database menolak foreign key yang tidak punya pasangan. JavaScript tidak. Jika ada kunjungan dengan `kodeDokter: "D9"`, `find` menghasilkan `undefined`, dan baris `d.nama` melempar `TypeError: Cannot read properties of undefined (reading 'nama')`. Jika data bisa tidak lengkap, cek hasil `find` sebelum dipakai.

2.6. Mengelompokkan Data (GROUP BY)

Untuk apa: membuat ringkasan per kelompok. Contohnya jumlah siswa dan total skor per kelas, atau jumlah produk per kategori. Hasilnya satu baris per **kelompok**, bukan satu baris per elemen.

```
const nilai = [
  { nama: "Ani",  kelas: "B", skor: 80 },
  { nama: "Budi", kelas: "A", skor: 70 },
  { nama: "Cici", kelas: "B", skor: 90 },
  { nama: "Dodi", kelas: "C", skor: 60 },
  { nama: "Eka",  kelas: "A", skor: 85 },
];
```

Listing 17: data nilai siswa

Di SQL cukup satu query:

```
SELECT kelas, COUNT(*), SUM(skor) FROM nilai GROUP BY kelas
```

Listing 18: versi SQL

Method array tidak punya `GROUP BY`. Pekerjaan itu dipecah menjadi dua langkah, dan setiap langkah memakai method yang sudah kamu kenal:

1. **Kumpulkan daftar kelompok yang unik**, seperti `SELECT DISTINCT kelas`. Hasilnya satu array string: `["B", "A", "C"]`.
2. **`map` di atas daftar kelompok itu.** Di dalam callback, ambil anggota kelompok dengan `filter`, lalu hitung ringkasannya.

Langkah 1 diperlukan karena aturan `map`: jumlah hasil sama dengan jumlah elemen yang di-`map`. Output yang diminta satu baris per kelas, jadi yang di-`map` harus array berisi kelas, bukan array `nilai` (yang akan menghasilkan 5 baris). Array berisi kelas tidak ada di data, jadi kita buat sendiri.

```
const kelasUnik = [];
for (const m of nilai) {
  if (!kelasUnik.includes(m.kelas)) kelasUnik.push(m.kelas);
}
// ["B", "A", "C"]
```

Listing 19: langkah 1: daftar kelas unik

Jalannya langkah 1. Urutan kelas mengikuti kemunculan pertamanya di data:

`m`	`m.kelas`	Sudah ada di `kelasUnik`?	`kelasUnik` sesudahnya
Ani	`"B"`	belum, `push`	`["B"]`
Budi	`"A"`	belum, `push`	`["B", "A"]`
Cici	`"B"`	sudah, dilewati	`["B", "A"]`
Dodi	`"C"`	belum, `push`	`["B", "A", "C"]`
Eka	`"A"`	sudah, dilewati	`["B", "A", "C"]`

```
const laporan = kelasUnik.map((k) => {
  const isi = nilai.filter((m) => m.kelas === k);              // anggota kelas k saja
  const totalSkor = isi.reduce((acc, m) => acc + m.skor, 0);   // SUM(skor)
  return `${k}: ${isi.length} siswa, total skor ${totalSkor}`; // COUNT(*) = isi.length
});
// ["B: 2 siswa, total skor 170", "A: 2 siswa, total skor 155", "C: 1 siswa, total skor 60"]
```

Listing 20: langkah 2: satu baris ringkasan per kelas

Perhatikan peran setiap bagian:

- `k` berisi **satu string kelas** per putaran `map`: `"B"`, lalu `"A"`, lalu `"C"`. Isinya bukan object siswa, jadi `k.kelas` tidak ada. Karena itu namanya `k`, bukan `m`.

- `map` **tidak** mengelompokkan. `map` hanya menyerahkan string kelas satu per satu.
- Yang mengelompokkan adalah `filter`. Di setiap putaran, `filter` memeriksa kelima siswa dan menyimpan yang `kelas`-nya sama dengan `k`. Ini sama dengan menjalankan `WHERE kelas = 'B'`, lalu `WHERE kelas = 'A'`, lalu `WHERE kelas = 'C'`.
- Setelah `isi` terbentuk, ringkasannya dihitung dengan cara biasa: `isi.length` untuk `COUNT(*)`, `reduce` untuk `SUM`.

Jalannya langkah 2:

Putaran	k	isi (hasil filter)	isi.length	totalSkor
1	"B"	Ani, Cici	2	170
2	"A"	Budi, Eka	2	155
3	"C"	Dodi	1	60

Versi `for`-nya memperlihatkan perulangan bersarang di dalamnya. Perulangan luar berjalan per kelas, dan perulangan dalam (pekerjaan `filter`) memeriksa semua siswa untuk setiap kelas:

```
const laporan2 = [];
for (const k of kelasUnik) {          // luar: setiap kelas (map)
  const isi = [];
  for (const m of nilai) {            // dalam: saring anggota kelas k (filter)
    if (m.kelas === k) isi.push(m);
  }
  let totalSkor = 0;
  for (const m of isi) totalSkor = totalSkor + m.skor;   // reduce
  laporan2.push(`${k}: ${isi.length} siswa, total skor ${totalSkor}`);
}
```

Listing 21: versi for dari langkah 2

Kesalahan umum:

- `map` dijalankan di array asli (`nilai.map(...)`). Hasilnya satu baris per siswa (5 baris), bukan per kelas.
- Variabel hasil dipakai di dalam perhitungannya sendiri, misalnya `hasil = kelasUnik.map((k) => hasil.length)`. Sisi kanan dijalankan sampai selesai dulu, baru disimpan ke `hasil`, jadi selama callback berjalan `hasil` masih bernilai awalnya. Jika awalnya `null`, muncul `TypeError: Cannot read properties of null (reading 'length')`. Data per kelompok ada di `isi`, bukan di `hasil`.
- Parameter `k` diperlakukan seperti object (`k.kelas`). `k` sudah berupa string kelas itu sendiri.

> ⚠ **Perhatian**
>
> JavaScript modern (ES2024, Node 21 ke atas) punya `Object.groupBy(array, callback)`. Hasilnya **object** yang key-nya nilai callback dan isinya array anggota, misalnya `{ B: [Ani, Cici], A: [Budi, Eka], C: [Dodi] }`. Object tidak punya `map`, jadi ubah dulu dengan `Object.entries(...)` menjadi array pasangan `[kelas, isi]`. Di TypeScript, fungsi ini butuh `lib` minimal `ES2024` di tsconfig. Kuasai dulu versi dua langkah di atas, karena polanya berlaku di bahasa apa pun.

2.7. for vs forEach

for / for...of	forEach
Bisa berhenti dengan `break` atau `return` dari fungsi	Tidak bisa berhenti. `return` di callback hanya melompati elemen itu
Cocok untuk mencari, atau berhenti saat kondisi terpenuhi	Cocok untuk aksi pada **semua** elemen

Bisa memakai `await` dengan benar (Modul 7)	Tidak menunggu `await` di dalam callback

`return` di dalam callback hanya keluar dari **callback itu**, bukan dari fungsi luarnya. `forEach` lalu memanggil callback lagi untuk elemen berikutnya.

```
[1, 2, 3].forEach((n) => {
  if (n === 2) return;   // hanya melompati n = 2
  console.log(n);
});
// 1
// 3
```

Listing 22: return di forEach tidak menghentikan perulangan

Jika perlu berhenti saat menemukan sesuatu, pakai `find`, `some`, atau `for` dengan `break`.

2.8. Membaca Pesan Error

Error berikut paling sering muncul saat memakai method array. Semuanya `TypeError`, artinya kamu memakai sebuah nilai dengan cara yang tidak cocok dengan tipenya.

Pesan error	Artinya	Penyebab umum
`Cannot read properties of undefined (reading 'nama')`	Kamu menulis `x.nama`, dan `x` bernilai `undefined`.	`find` tidak menemukan apa-apa. Membaca properti dari array, bukan elemennya. Index di luar batas.
`x.map is not a function` (atau `join`, `filter`, dan lainnya)	`x` ada, tetapi bukan array, jadi tidak punya method itu.	`x` adalah hasil `find` (object), hasil `join` (string), atau hasil `reduce` (number).
`Reduce of empty array with no initial value`	`reduce` dipanggil pada array kosong tanpa nilai awal.	Lupa argumen kedua `reduce`.

Cara melacaknya selalu sama: lihat nomor baris di pesan error, cari nilai **di sebelah kiri titik** (`x` pada `x.nama`), lalu `console.log` nilai itu tepat sebelum baris tersebut. Biasanya bentuknya berbeda dari yang kamu kira.

Catatan

Saat ditanya “kenapa pakai `filter` dan `map`, bukan `for`?”, jawaban yang baik: method array menunjukkan **niat** (menyaring, mengubah, menjumlahkan) dan tidak mengubah data asli. `for` tetap tepat ketika kamu perlu berhenti lebih awal atau logikanya terlalu rumit untuk satu callback.

Coba Sendiri

Kerjakan di fungsi `cobaSendiri()` pada file soal `Jawaban/soal-js3.js` (sudah tersedia di starter). Sebelum menjalankan, **tulis tebakanmu** sebagai komentar di ujung setiap baris `console.log`. Jalankan bagian ini saja dengan `node Jawaban/soal-js3.js coba`, lalu bandingkan.

```
const angka = [5, 25, 100, 1];
console.log(angka.sort());
console.log(angka);
console.log(pasien.map((p) => p.umur > 18));
console.log(pasien.filter((p) => p.umur > 18).length);
console.log(pasien.find((p) => p.umur > 18).nama);
console.log([].reduce((a, b) => a + b, 0));
const hasil = pasien.forEach((p) => p.nama);
console.log(hasil);
```

1. Tebakan mana yang salah? Jelaskan penyebabnya.
2. Apa beda hasil `map` dan `filter` pada baris ketiga dan keempat?
3. Hapus `, 0` pada baris `reduce`, lalu jalankan. Catat error-nya dan jelaskan.
4. Dengan chaining, tampilkan nama pasien poli Umum dalam satu string dipisah koma: `Budi, Joko`. Petunjuk: method `join(", ")`.

Jawab pertanyaan di atas di bawah baris `// JAWABAN Coba Sendiri N` pada file soal.